

TP4 - Indexation de Texte par ABR

Timothée POUPINEAU & Loudiern THARON

18 décembre 2025

1 Structures et Fonctions Supplémentaires

1.1 Structures Supplémentaires

En plus des structures demandées (`T_Position`, `T_Noeud`, `T_Index`), nous avons implémenté :

- **T_MotOrdre** (B.6 et B.7) : Liste chaînée stockant un mot et son ordre dans la phrase. Permet de reconstruire une phrase en ordre croissant.

Justification : Ces structures évitent des parcours répétés de l'ABR et permettent de trier les mots efficacement pour reconstituer les phrases dans le bon ordre.

1.2 Fonctions Auxiliaires

- `initialiserIndex()` : Initialise un index vide
- `convertirMinuscules()` : Ignore la casse (voiture = Voiture = VOITURE)
- `libererPositions()`, `libererNoeud()`, `libererIndex()` : Libération mémoire
- `afficherNoeudInfixe()` : Parcours infixe pour affichage alphabétique
- `ajouterMotOrdre()` : Insertion triée dans une liste de mots ordonnés (B.6)
- `collecterMotsPhrase()` : Collecte les mots d'une phrase donnée (B.6)
- `afficherPhrase()`, `libererMotsOrdres()` : Affichage et libération (B.6)
- `compterPhrasesMax()` : Trouve le numéro de phrase maximum (B.7)

Justification : Modularité du code, réutilisation des fonctions et gestion rigoureuse de la mémoire dynamique.

2 Complexité des Fonctions

Notations : n = mots distincts, h = hauteur ABR, p = positions/mot, m = mots totaux, k = longueur mot, s = nb phrases.

Cas moyen : $h = O(\log n)$. **Pire cas** (arbre dégénéré) : $h = O(n)$.

2.1 Fonctions Demandées (B.1 à B.7)

Fonction	Complexité	Explication
ajouterPosition()	$O(p)$	Parcours de la liste pour insertion triée
ajouterOccurrence()	$O(h + p)$	Descente dans l'ABR + insertion position
indexerFichier()	$O(m \times h)$	Lecture fichier + m insertions dans l'ABR
afficherIndex()	$O(n \times p)$	Parcours infixe + affichage positions
rechercherMot()	$O(h)$	Recherche binaire dans l'ABR
afficherOccurrencesMot()	$O(k \times n \times p)$	k occurrences $\times$ parcours ABR/phrased
construireTexte()	$O(s \times n \times p)$	s phrases $\times$ parcours ABR par phrase

2.2 Fonctions Auxiliaires

Fonction	Complexité	Explication
initialiserIndex()	$O(1)$	Initialisation des 3 champs
convertirMinuscules()	$O(k)$	Parcours de la chaîne (k = longueur)
afficherNoeudInfixe()	$O(n \times p)$	Parcours récursif infixe complet
ajouterMotOrdre()	$O(m)$	Insertion triée (m = mots/phrased)
collecterMotsPhrase()	$O(n \times p)$	Parcours ABR + vérification positions
afficherPhrase()	$O(m)$	Parcours liste de mots
libererMotsOrdres()	$O(m)$	Libération liste chaînée
compterPhrasesMax()	$O(n \times p)$	Parcours ABR + recherche max phrase
libererPositions()	$O(p)$	Libération liste positions
libererNoeud()	$O(n)$	Libération récursive sous-arbre
libererIndex()	$O(n)$	Appel libererNoeud sur racine

3 Choix d'Implémentation

- **Casse** : Conversion systématique en minuscules via `tolower()`.
- **Ordre** : `strcmp()` pour l'ordre lexicographique dans l'ABR.
- **Positions triées** : Insertion en $O(p)$ mais évite des tris ultérieurs.
- **Mémoire** : Libération complète via `libererIndex()` à la fin du programme.
- **B.6 optimisé** : Pour chaque occurrence, parcours unique de l'ABR pour collecter les mots de la phrase concernée, insertion triée par ordre.
- **B.7 réutilise B.6** : Utilise `collecterMotsPhrase()` pour chaque phrase, évitant la duplication de code.